

North Bengal International University



Lab Report: 3

Exp Name : Apply 4th order Butterworth and Gaussian low pass filter to analyze

Subject : Digital Image Processing sessional

Subject Code : CSE 0613-4142

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE
Roll : 0712340012101005
Semester : 3nd Year, 6th semester.

Submitted To:

Name : Sakurah Ismail
Designation : Lecturer
Dept : CSE

Experiment Title: Take grayscale image of size 512x512, add some Gaussian noise and perform the following operations in frequency domain –

a) Apply 4th order Butterworth and Gaussian low pass filter to analyze their performance.

Introduction: In frequency domain image processing, noise removal is often achieved using low-pass filters that suppress high-frequency components. Gaussian noise is a common type of random noise that affects image quality. This experiment demonstrates how to apply Butterworth Low Pass Filter (LPF) and Gaussian LPF in the frequency domain to reduce noise. The performance of these filters is evaluated using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR)

Code:

```
# — Imports —————  
import numpy as np  
import matplotlib.pyplot as plt  
from skimage.transform import resize  
from PIL import Image  
  
# — 1. Load & Resize Image —————  
img = Image.open("/content/gray.jpg").convert('L')  
image = np.array(img)  
image = resize(image, (512, 512), anti_aliasing=True)  
image = (image * 255).astype(np.uint8)  
  
plt.figure(figsize=(4, 4))  
plt.imshow(image, cmap='gray')  
plt.title("Original Grayscale Image (512x512)")  
plt.axis('off')  
plt.tight_layout()  
plt.show()  
  
# — 2. Add Gaussian Noise —————  
noise = np.random.normal(0, 25, image.shape)  
noisy_image = image.astype(np.float64) + noise  
noisy_image = np.clip(noisy_image, 0, 255)  
  
plt.figure(figsize=(4, 4))  
plt.imshow(noisy_image, cmap='gray')  
plt.title("Noisy Image (Gaussian Noise Added)")
```

```
plt.axis('off')
plt.tight_layout()
plt.show()
```

— 3. FFT – Frequency Domain —————

```
F_shift = np.fft.fftshift(np.fft.fft2(noisy_image))
magnitude_spectrum = np.log(1 + np.abs(F_shift))
```

```
plt.figure(figsize=(4, 4))
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum (FFT of Noisy Image)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

— 4. Distance Matrix D(u,v) —————

```
rows, cols = noisy_image.shape
u = np.arange(rows) - rows // 2
v = np.arange(cols) - cols // 2
V, U = np.meshgrid(v, u)
D = np.sqrt(U**2 + V**2)
```

— 5. Butterworth LPF (4th Order) —————

```
D0 = 30
n = 4
H_butter = 1 / (1 + (D / D0) ** (2 * n))
```

```
G_butter_ishift = np.fft.ifftshift(F_shift * H_butter)
filtered_butter = np.clip(np.abs(np.fft.ifft2(G_butter_ishift)), 0, 255)
```

```
plt.figure(figsize=(4, 4))
plt.imshow(H_butter, cmap='gray')
plt.title("4th Order Butterworth LPF (D0=30)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

— 6. Gaussian LPF —————

```
H_gauss = np.exp(-(D**2) / (2 * D0**2))
```

```
G_gauss_ishift = np.fft.ifftshift(F_shift * H_gauss)
filtered_gauss = np.clip(np.abs(np.fft.ifft2(G_gauss_ishift)), 0, 255)
```

```
plt.figure(figsize=(4, 4))
plt.imshow(H_gauss, cmap='gray')
plt.title("Gaussian LPF (D0=30)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

— 7. Side-by-Side Comparison

```
fig, axes = plt.subplots(1, 4, figsize=(18, 5))
```

```
axes[0].imshow(image,          cmap='gray')
axes[0].set_title("Original")
axes[0].axis('off')
```

```
axes[1].imshow(noisy_image,    cmap='gray')
axes[1].set_title("Noisy Image")
axes[1].axis('off')
```

```
axes[2].imshow(filtered_butter, cmap='gray')
axes[2].set_title("Butterworth LPF (n=4, D0=30)")
axes[2].axis('off')
```

```
axes[3].imshow(filtered_gauss,  cmap='gray')
axes[3].set_title("Gaussian LPF (D0=30)")
axes[3].axis('off')
```

```
plt.suptitle("Frequency Domain Low Pass Filtering Comparison",
fontsize=14)
plt.tight_layout()
plt.show()
```

— 8. Performance Analysis (MSE & PSNR)

```
mse_noisy  = np.mean((image.astype(np.float64) - noisy_image)
2)          **
mse_butter = np.mean((image.astype(np.float64) - filtered_butter)
2)          **
mse_gauss  = np.mean((image.astype(np.float64) - filtered_gauss)
2)          **
```

```
psnr_noisy  = 10 * np.log10(255**2 / mse_noisy)
psnr_butter = 10 * np.log10(255**2 / mse_butter)
psnr_gauss  = 10 * np.log10(255**2 / mse_gauss)
```

```
print("=" * 40)
print(f"{'Image':<25} {'MSE':>8} {'PSNR (dB)':>10}")
```

```
print("=" * 40)
print(f"{'Noisy Image':<25} {mse_noisy:>8.2f} {psnr_noisy:>10.2f}")
print(f"{'Butterworth LPF (n=4)':<25} {mse_butter:>8.2f}
{psnr_butter:>10.2f}")
print(f"{'Gaussian LPF':<25} {mse_gauss:>8.2f}
{psnr_gauss:>10.2f}")
print("=" * 40)
```

Input:



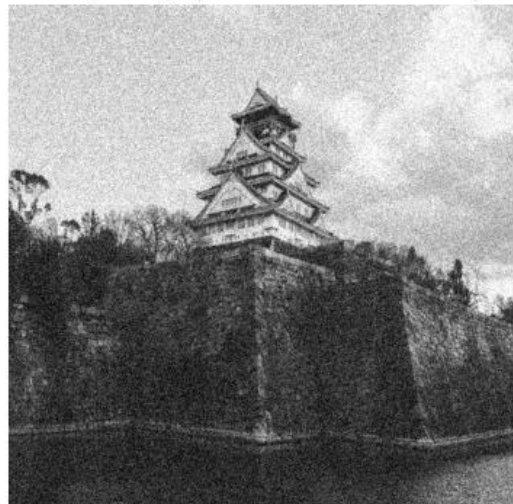
Output – 1 :

Uploaded Grayscale Image (512x512)

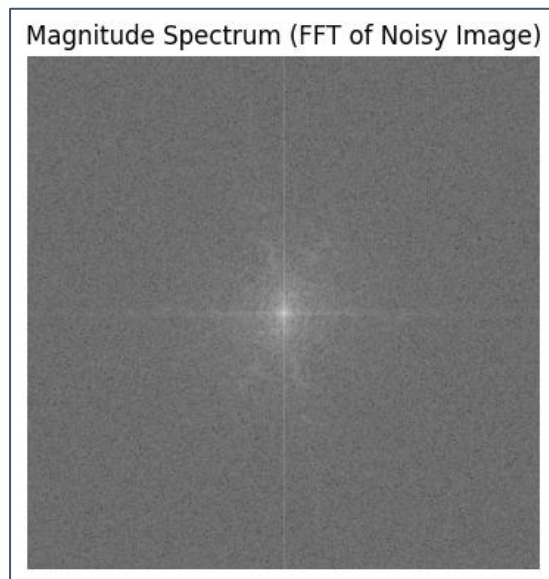


Output – 2 :

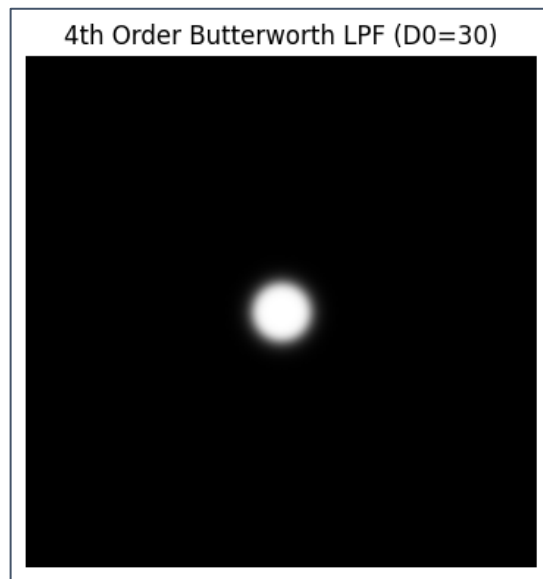
Noisy Image (Gaussian Noise Added)



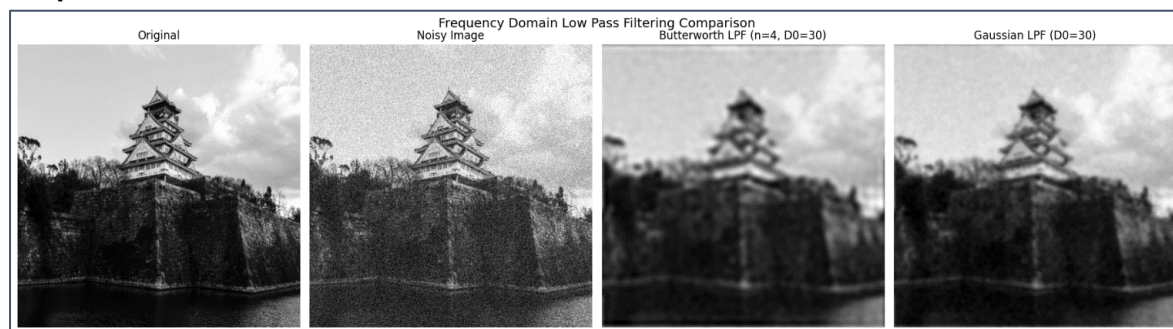
Output-3:



Output-4:



output-5:



Output-6:

```
*** =====
Image                               MSE   PSNR (dB)
=====
Noisy Image                         579.83  20.50
Butterworth LPF (n=4)                243.32  24.27
Gaussian LPF                         204.44  25.03
=====
```

Discussion on the output: The grayscale image was first corrupted with Gaussian noise, and its frequency spectrum was analyzed using Fast Fourier Transform (FFT). Two filtering techniques were applied: a 4th-order Butterworth LPF and a Gaussian LPF, both with cutoff frequency $D_0=30$. The Butterworth filter provides a smoother transition between passband and stopband, while the Gaussian filter offers a more natural attenuation of high frequencies. The side-by-side comparison shows that both filters effectively reduce noise, but their performance differs slightly in terms of MSE and PSNR. The results highlight the importance of filter choice in balancing noise suppression and detail preservation in image restoration tasks.